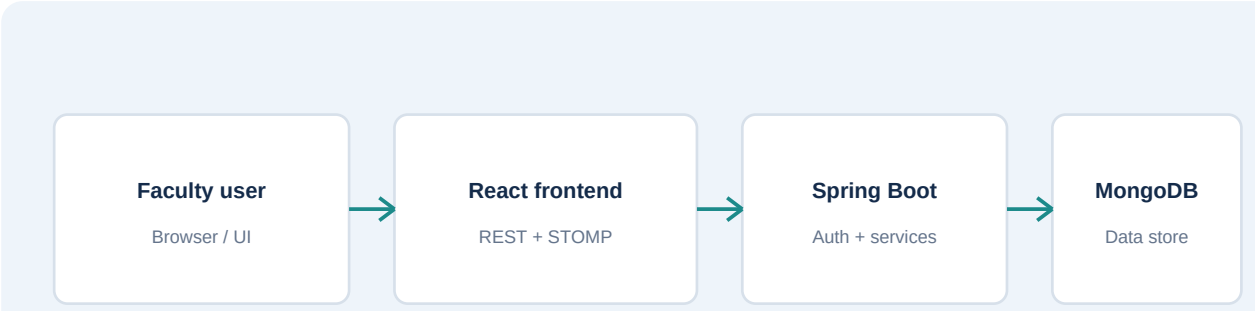
FACULTY CONNECT

A Faculty-Focused Communication and Collaboration Platform

PROJECT RESEARCH PAPER



Conceptual architecture for the current prototype

Connect. Collaborate. Communicate.

Academic Project | IPS Academy
Department of Computer Science & Information Technology

Prepared by: Faculty Connect Project Team
Submitted to: Project Coordinator

This paper presents the project motivation, proposed design, current implementation scope, and a plan for evaluating the prototype. Implementation status should be reviewed before submission.

Abstract

Communication and coordination are recurring activities in higher education institutions. Faculty members may need to discuss academic matters, coordinate within departments, exchange documents, and communicate updates with colleagues. When these activities are distributed across general-purpose messaging tools, email, and informal channels, conversations and resources may become fragmented. Faculty Connect is proposed as a faculty-focused communication and collaboration platform that brings key interactions into a common workspace. The system is designed around authenticated faculty accounts, faculty profiles, one-to-one conversations, academic groups, message history, and real-time messaging. The prototype uses React.js for the frontend and Java with Spring Boot for backend services, MongoDB for persistence, JWT-based authentication, and WebSocket/STOMP for real-time message delivery. This paper explains the problem motivation, objectives, comparison with existing platforms, system architecture, implementation status, limitations, and an evaluation plan. The contribution is not the invention of messaging itself; rather, it is a focused prototype and design approach for organizing communication around faculty identity and academic collaboration. The practical value should be assessed through security testing, usability feedback, and pilot use by faculty members.

Keywords: Faculty communication; academic collaboration; real-time messaging; higher education; Spring Boot; MongoDB; JWT; WebSocket; STOMP.

Contents

- 1. Introduction
- 2. Problem Statement
- 3. Aim and Objectives
- 4. Existing Solutions and Rationale
- 5. Proposed System and Core Features
- 6. System Architecture and Technology Stack
- 7. Data Model Overview
- 8. Security and Privacy Considerations
- 9. Current Implementation Status
- 10. Testing and Evaluation Plan
- 11. Expected Benefits
- 12. Limitations and Future Work
- 13. Conclusion
- References
- Appendix A — Short Presentation Summary

1. Introduction

Colleges depend on regular coordination among faculty members for teaching, departmental activities, academic planning, document exchange, and institutional updates. Digital communication tools make coordination faster, but a college community may use several different channels for different purposes. Faculty members may need to move between private chats, group chats, email, and shared files to locate relevant information.

Faculty Connect is a faculty-focused platform intended to organize these activities in a single application. The system focuses on faculty identity and academic context: who the faculty member is, which department they belong to, which conversations they participate in, and which groups support a particular subject or department. The project is being developed as a working software prototype, with frontend and backend work progressing in parallel.

2. Problem Statement

General-purpose communication applications are useful and widely adopted, but a college may still need a purpose-configured workspace for faculty communication. Distributed communication can make it harder to keep academic conversations organized, connect discussions to departments or subjects, and manage access according to faculty membership. The project investigates whether a focused platform can bring these workflows together while preserving authenticated access and controlled conversation membership.

This is a project motivation and design hypothesis, not a claim that every college experiences the same problem. The scale of the problem and usefulness of the proposed solution should be validated with interviews, surveys, or a pilot deployment.

3. Aim and Objectives

- Develop a common digital workspace for authorized faculty members.
- Provide secure registration and login using password hashing and JWT authentication.
- Support one-to-one conversations and retrieval of previous messages.
- Allow faculty members to create and manage academic groups and their membership.
- Provide real-time messaging using WebSocket and STOMP.
- Maintain faculty profile information relevant to academic collaboration.
- Design the frontend and backend as modular components that can be extended with notifications, file sharing, announcements, and delivery/read receipts.
- Evaluate the prototype through functional tests, security checks, and feedback from representative faculty users.

4. Existing Solutions and Rationale

The project does not assume existing applications lack messaging or collaboration features. WhatsApp provides group messaging and community features; Slack organizes collaboration around channels and supports file sharing; Microsoft Teams for Education supports chats, channel posts, announcements, and shared files. These capabilities are documented by their providers [1–6].

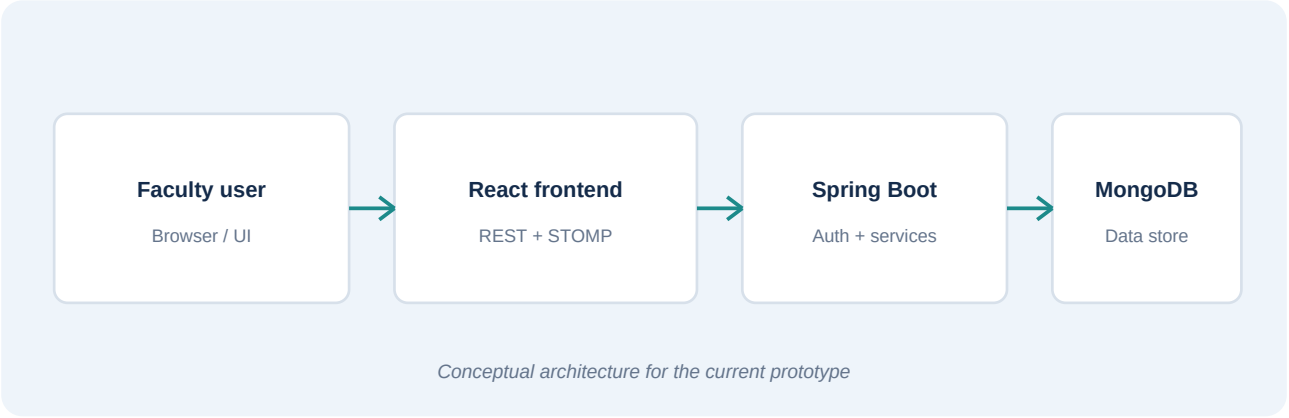
The reason to develop Faculty Connect is not that it can automatically outperform established products. Its proposed value is a narrower scope, an academic-first information structure, and the opportunity to tailor access, groups, and workflows to one institution. Whether this focus is more convenient than configuring an existing platform remains an evaluation question.

5. Proposed System and Core Features

- Authentication and access control: Faculty registration and login, BCrypt password hashing, JWT authentication, and protected APIs.
- Faculty profiles: View and update name, department, designation, subjects, profile image, and bio.
- One-to-one conversations: Create or retrieve a conversation between two faculty members and prevent duplicate conversations for the same pair.
- Messaging and history: Send text messages through REST APIs, persist them in MongoDB, and retrieve conversation history.
- Real-time communication: Use WebSocket/STOMP so connected participants can receive new messages without repeated polling.
- Academic groups: Create groups, view groups accessible to the logged-in faculty member, and add/remove members subject to authorization.
- Future scope: Group real-time chat, announcements, document uploads, notifications, online presence, typing indicators, and delivery/read receipts.

6. System Architecture and Technology Stack

The system follows a frontend/backend architecture. The React frontend calls REST endpoints through Axios and uses a STOMP client for real-time chat. The Spring Boot backend validates requests, authenticates users, enforces conversation/group membership rules, and stores application data in MongoDB.



Conceptual architecture for the current prototype

- Frontend: React.js, Vite, JavaScript, Tailwind CSS v4, React Router DOM, Redux Toolkit, React Redux, Axios, and Lucide React.
- Backend: Java 21, Spring Boot, Spring Web, Spring Data MongoDB, Spring Security, BCrypt, JJWT 0.12.6, and Spring WebSocket/STOMP.
- Database: MongoDB database named faculty_connect.
- REST API base URL during local development: http://localhost:8080.
- WebSocket endpoint during local development: ws://localhost:8080/ws.
- STOMP send destination: /app/chat.send; conversation subscription: /topic/conversation/{conversationId}.

7. Data Model Overview

The current backend uses MongoDB document collections rather than a relational database schema. The main collections implemented in the project are:

- faculties: faculty ID, name, email, password hash, department, designation, subjects, profile details, role, and timestamps.

- conversations: unique conversation ID, two participant email identifiers, and timestamps.
- messages: message ID, conversation ID, sender email, receiver email, content, message type, status, and timestamps.
- groups: group ID, group name, description, department/subject context, creator, member identifiers, and timestamps.

Relationship summary: one faculty member can participate in many conversations and groups; each one-to-one conversation contains two faculty participants; a conversation can contain many messages; a group can contain multiple faculty members. In this MongoDB model, participant/member email identifiers are stored in arrays. This is a document-model overview, not a relational ER diagram.

8. Security and Privacy Considerations

- Passwords are hashed using BCrypt rather than stored as plain text.
- JWT tokens identify the authenticated faculty member for protected API requests and STOMP CONNECT frames.
- The backend derives message sender identity from the authenticated principal rather than trusting a sender ID supplied by the client.
- The backend checks conversation membership before reading or sending messages and checks membership before subscribing to a conversation topic.
- Profile updates should not allow users to change protected fields such as faculty ID, email, password, or role through the profile-update endpoint.
- The local-development setup is not production-ready without reviewing secret management, HTTPS/WSS, database access control, rate limiting, logging, backups, and deployment configuration.

9. Current Implementation Status

Based on the project development notes, the backend includes faculty registration/login, JWT-protected APIs, profile retrieval/update, one-to-one conversation management, group management and membership operations, REST message sending/history, and a WebSocket/STOMP messaging layer with JWT authentication and conversation subscription authorization. The duplicate conversation lookup fix was followed by a reported successful Maven test run. The live WebSocket end-to-end flow should still be manually verified with two authorized participants and one non-participant account before describing it as fully validated.

Online/offline presence, typing indicators, reliable delivered/read receipts, notifications, file/document upload, announcements, and group real-time messaging remain future or separate implementation work unless subsequently confirmed as completed. Frontend UI and API integration are being developed in parallel. Update this section to reflect the actual state on the day of submission.

10. Testing and Evaluation Plan

A successful compilation is necessary but does not alone demonstrate that the full application works. The prototype should be evaluated using the following test plan:

- Authentication: valid login succeeds; invalid credentials fail; protected APIs reject missing, invalid, or expired tokens.
- Conversation creation: the same pair receives the existing conversation rather than duplicate records; self-conversation is rejected.
- Message access control: participants can read/send messages; non-participants are denied.
- WebSocket behavior: two participants connect with valid JWTs, subscribe to the same conversation, and verify that a sent message is persisted and received by the other connected client.

- Subscription security: a third faculty member who is not a participant cannot subscribe to the conversation topic.
- Group authorization: creation, listing, member addition/removal, and deletion follow intended access rules.
- Usability: representative faculty users complete common tasks; record task completion, time, and qualitative feedback.

Suggested evaluation questions: Can faculty members find the correct colleague quickly? Are department/subject groups easy to understand? Can users locate a previous message? Do users prefer this focused workflow to their current process? Collect and report responses honestly; do not fabricate results.

11. Expected Benefits

- A single interface for the communication workflows included in the prototype.
- A clearer connection between faculty identity and academic group membership.
- A modular foundation that can be extended with announcements, document sharing, notifications, and presence features.
- A practical learning project demonstrating frontend/backend integration, authentication, document-database modeling, API design, and real-time messaging.

These are expected benefits, not measured outcomes. Claims of improved productivity, reduced response time, or higher user satisfaction should be supported by user testing or other evidence.

12. Limitations and Future Work

The current prototype is not yet equivalent to a mature enterprise collaboration platform. It needs end-to-end validation, a finished frontend, and further features such as group real-time messaging, document upload, announcements, notifications, presence, typing indicators, and delivery/read receipts. Production deployment would also require stronger operational controls, monitoring, backups, secure secret storage, and a clear institutional onboarding/offboarding process. Future evaluation should compare Faculty Connect with existing tools using the same task scenarios and representative users.

13. Conclusion

Faculty Connect explores a focused digital workspace for communication among college faculty members. Instead of claiming to replace established platforms or offer entirely new messaging capabilities, the project brings together faculty authentication, profiles, one-to-one conversations, academic group management, persistent message history, and a real-time messaging design. React, Spring Boot, MongoDB, JWT, and WebSocket/STOMP provide a modular technical foundation. The next step is to validate the end-to-end experience with real faculty users, complete remaining features, and use measurable feedback to determine whether the academic-first approach offers practical value for a college.

References

- [1] WhatsApp, “Discover WhatsApp Group Messaging Features.” https://www.whatsapp.com/groups
- [2] WhatsApp Help Center, “About communities.” https://faq.whatsapp.com/495856382464992
- [3] Slack, “Slack Features: Team Collaboration Tools.” https://slack.com/features
- [4] Slack, “Document Sharing.” https://slack.com/features/document-sharing
- [5] Microsoft Support, “Meetings, chats, and posts in class teams.” https://support.microsoft.com/en-us/teams/education/quick-start/meetings-chats-and-posts-in-class-teams
- [6] Microsoft Education, “Microsoft Teams for Education.” https://www.microsoft.com/en/education/products/teams

• [7] Microsoft Learn, “Standard channels in Microsoft Teams.” https://learn.microsoft.com/en-us/microsoftteams/standard-channels

Appendix A — Short Presentation Summary

Faculty Connect is a faculty-focused communication and collaboration prototype. It investigates how a college might organize faculty messaging around authenticated faculty accounts, departments, subjects, and controlled group membership. The backend uses Java Spring Boot, MongoDB, JWT, and WebSocket/STOMP; the frontend uses React and related libraries. Existing products already provide powerful messaging and collaboration, so the project's value must be judged by its focused workflow and feedback from faculty users. Demonstrate only features that have been tested and clearly identify planned features.